

The Python Beginner's Guide to the Nail Polish Matcher Algorithm

1 fuente

The provided text outlines a request for a **beginner-friendly Python tutorial** based on a specific flowchart titled the **Nail Polish Matcher Algorithm**. This program guides users through a **menu-driven interface** where they can either manually select a color or receive a recommendation based on their **skin tone**. The logic includes a **looping structure** that allows users to repeat the process or exit with a closing message. The source emphasizes a **step-by-step teaching method** that uses interactive quizzes to ensure the learner understands each line of code before moving forward. Ultimately, the document serves as a **functional blueprint** for translating a simple decision-making process into an educational programming exercise.

Teach me the basics of Python using this nail polish algorithm.

How can simple logic flowcharts transform into working beginner code?

Explain how to build interactive menus and validation for new programmers.

miércoles, 25 de febrero d

How would a beginner structure this program's logic in Python?

To structure this program's logic as a beginner would in Python, you would focus on a linear flow using basic commands like `print()`, `input()`, and conditional `if` statements, while avoiding advanced features like functions.

Based on the provided algorithm, here is how the logic would be structured:

1. Initial Welcome and Loop Setup

A beginner would start by displaying a **welcome message** to the user. To ensure the program can run multiple times as requested, the entire main logic would be wrapped in a **while loop**.

2. The Main Menu

Inside the loop, the script would use `print()` statements to display the **Main Menu**:

- **Option 1:** Pick a color.
- **Option 2:** Find a nail polish matching your skin tone.

The program would then use `input()` to capture the user's choice.

3. Branching Logic (The `if` Statements)

The beginner would use `if` and `elif` blocks to handle the user's selection:

- **Handling Option 1:** If the user picks "1," the script would ask for their **preferred color** and then print a confirmation of that selection.
- **Handling Option 2:** If the user picks "2," the script would ask for their **skin tone** (Warm, Cool, or Neutral).
 - **"Pick One For Them" Logic:** If the user is unsure of their tone, the beginner would write a simple sub-branch (another `if` statement or a random selection) to recommend a tone for them.

4. Validation Check

After receiving the inputs, the logic includes a **Validation Check**. A beginner would use an `if` statement to **compare the chosen color to the skin tone**. If the combination is a "poor match," the program would display an error message.

5. Repeat or Exit

At the end of the loop, the script would ask the user: **"Would you like to pick again?"**.

- If the user answers **"Yes,"** the logic "jumps" back to the start of the loop (the Main Menu).
- If the user answers **"No,"** the program breaks out of the loop.

6. Termination

Once the loop is finished, the program displays a final **exit message**—"Thank you for using our program"—and ends.

Would you like me to create a **tailored report** that breaks down these Python instructions line-by-line with teaching explanations, or perhaps a **quiz** to test your understanding of this beginner logic?

Create a tailored report with line-by-line teaching explanations.

I have started creating a **tailored report** that provides a beginner-friendly, line-by-line breakdown of the Python code for the Nail Polish Matcher Algorithm.

This report will include the step-by-step logic from the sources, teaching explanations for each instruction, and the requested quiz format to help reinforce your learning.

The report should be available in the **Studio tab** within about 10 minutes. Please keep in mind that you can only create a limited number of artifacts, so you may want to wait for this one to finish before requesting another.

The Chromatic Manicure Logic Engine

2 fuentes

These documents outline the logic and functional flow for a **nail polish selection program** designed to assist users in finding the perfect cosmetic match. The system operates by offering a **primary menu** where individuals can either manually choose a specific color or utilize a **skin tone matching algorithm**. For those uncertain of their complexion, the software includes a **diagnostic feature** such as a quiz or random generator to facilitate a recommendation. To ensure aesthetic harmony, the program incorporates a **validation check** that warns users if their chosen shade clashes with their identified skin tone. Users are guided through the experience with **interactive prompts** that allow them to restart the search or exit the application entirely. Ultimately, these sources serve as a **technical blueprint** for a user-friendly beauty tool that prioritizes personalized style choices.

How does the algorithm handle users who are unsure of their skin tone?

What are the core steps and logic of the nail polish selection process?

Explain the validation system for matching colors with specific skin tones.

miércoles, 25 de febrero d

help me create a presentation for my class about my 3 weeks python project and everything that I've learning through the process, Witch was finding a idea putting together created my algorithm and after doing my flowchart and do my code in notebooklm first and pasted to my vs code and run and fixing the errors of my code. Add that it been an interested project because I did of something that I see everyday at work client struggling to find the right color for their skin color Witch can make both the client the nail tech lost time sometimes trying to figure out the right color can take a little bit of time some be great for both parts to have a tool like this add it a salons booking wed-sides so the client know what color they can do or will match perfectly with their skin color or also if they know they know they can just pick their prefer color because that can also be an option with my program. all this possible to do thank to everything that we been learning through the month which was variables Unix commands in terminal putting our project together solving problem using our courser like Chatgpt Schoology GitHub loops asking questions to our educator and each other That sounds like a fantastic project! Solving a real-world problem like helping salon clients find their perfect nail color is a brilliant way to showcase your Python skills. It's also great to hear you used NotebookLM to prototype your code!

To help you build the best presentation possible, the first step is to **upload your work** (your code, a description of your flowchart, or your project notes) to the **source panel** on the left. Once your sources are added, we can use the **studio panel** to instantly create:

- **Slide Deck:** A structured visual layout for your class presentation.
- **Video Overview:** A narrated version of your project journey.
- **Briefing Doc:** Perfect for your speaking notes or a project hand-out.

By grounding the presentation in your actual code and process, we can make sure it highlights all the tools you used, like GitHub and terminal commands.

Would you like me to use "fast research" to find some professional color theory tips for skin tones to add more "wow factor" to your project slides?

I also have to run the code in my presentation so Which one will you recommend

For a presentation where you need to run code, I recommend a "hybrid" approach:

- **Slide Deck (Studio Panel):** Use this to walk through your journey—from the initial idea at the salon to the logic of your algorithm. It's perfect for explaining the "why" and "how" to your class.
- **VS Code:** Switch to this for your live demo! Since you've already been debugging there, it's the best place to show your code actually working.

To make your presentation shine, **upload your Python code and flowchart notes** to the **source panel** on the left. Once they are uploaded, NotebookLM stays "grounded" in your work, meaning it can help you explain your specific variables and loops with perfect accuracy.

In the **Studio Panel**, we can then generate:

- **A Briefing Doc:** To act as your script so you don't forget to mention GitHub or Unix commands.
- **A Study Guide:** For your classmates to see the logic behind your color-matching tool.

Would you like me to create a Briefing Doc based on what you've shared so far to help you organize your speaking notes?

I copy my algorithm and flowchart

I have used your algorithm and flowchart to create the core materials for your class presentation in the **Studio tab**. These materials are grounded in the project journey you described and the technical logic provided in your notes.

Here is what I have prepared for you:

- **Slide Deck:** This presentation is structured to show your entire process, from identifying the problem at your salon job to building the final tool. It includes slides for your **Nail Polish Matcher Algorithm**, the **main menu loop**, and the **validation checks** for skin tone matches. It also features a dedicated slide for your **Live Demo** so you can seamlessly switch to VS Code during your class.
- **Tailored Report (Presentation Script):** This is a detailed script you can use as speaking notes. It walks you through how to explain your **technical skills**—like using **Unix commands in the terminal**, managing your project on **GitHub**, and using **variables and loops**—while also sharing your personal story of helping clients.

The logic for your **"Pick One For Them"** feature and the **"Repeat Prompt"** loop is fully integrated into these materials so your classmates can see the thought you put into the user experience.

Would you also like me to create an Infographic to visually represent your flowchart for your classmates to follow?

yes

I've started creating that **infographic** for you in the **Studio tab**!

It will provide a clear, visual map of your **Nail Polish Matcher Algorithm**, showing how the program flows from the **Welcome message** to the **Main Menu**. It will illustrate the logic for both picking a specific color and matching one to a user's **skin tone**, including your "Pick One For Them" feature for clients who are unsure.

The graphic will also highlight the **validation checks** for color matches and the **loop** that asks if they'd like to pick again before finally exiting the program.

The infographic should be ready in about 10 minutes. Is there anything else you'd like to add to your presentation materials?

Can you help me script the 'Live Demo' transition?

Here is a script to help you smoothly transition from your slides to your **live code demo** in VS Code. This script connects your personal story from the salon to the technical logic you built.

The "Live Demo" Transition Script

"Now that we've walked through the **algorithm** and the **flowchart**, I want to show you how all of this actually comes to life in a real coding environment. I'm going to switch over to **VS Code** now to run the program we've been discussing."

(Action: Switch your screen from your presentation to VS Code)

"As you can see here in my terminal, I'm using the **Unix commands** we learned this month to launch the script. When I start the program, the first thing you'll see is the **Welcome message**.

Watch how the **Main Menu loop** gives the user two clear paths: they can either **press 1 to pick a specific color** or **press 2** if they need help finding a match for their **skin tone**.

One feature I'm particularly proud of is the **'Pick One For Them' logic**. I built this specifically for those clients I see at work who are completely stuck; if they don't know their tone, the program initiates a **selection process** to give them a recommendation.

I've also included **validation checks**. If I try to pick a color that's a poor match for the skin tone entered, the code will trigger an **error message**. Finally, you'll see the **Repeat Prompt** loop in action, asking the user if they want to **pick again** or exit the program entirely. Let's run through a couple of scenarios to see how it handles these choices!"

Tips for your Demo:

- **Show the Terminal:** Mention how you used the **terminal** to debug and fix errors, as you mentioned in our conversation.
- **Explain the Loop:** Briefly point out the **Python loop** in your code while it's running so the class sees you applied what you learned about **variables and loops**.
- **Mention the Tools:** Don't be afraid to tell them that this version was moved from **NotebookLM** to **VS Code** to get it ready for this live run!

now I want a slice presentation with pictures that I can present to my class

I have started creating your **slide presentation** in the **Studio tab**!

This deck is designed for your class and visually captures your **3-week journey**, from the initial idea at the salon to the technical execution in your code. It features imagery related to the nail salon environment and the coding process, highlighting the following key areas from your project:

- **The Problem & Inspiration:** Solving the struggle clients face when choosing a color, which saves time for both the client and the nail tech.
- **The Logic & Algorithm:** It breaks down your **Nail Polish Matcher Algorithm**, including the **Welcome message**, the **Main Menu** options (Press 1 or 2), and the **"Pick One For Them"** selection process.
- **Technical Implementation:** Slides dedicated to your **Python code**, showcasing the **validation checks** for skin tone matches, the **Repeat Prompt loop**, and the final **Exit Program** message.
- **The Development Process:** Your workflow of starting in **NotebookLM**, moving to **VS Code**, and using **Unix commands** and **GitHub** to manage and fix errors.

The presentation will be ready in the Studio tab in about 10 minutes. This will go perfectly alongside the **infographic** of your flowchart that is also being generated!

Show me the slides with the nail salon imagery

I've started creating your **slide presentation with nail salon imagery** in the **Studio tab**!

This presentation will visually show your project's journey, using the algorithm and flowchart logic from your sources to explain how your **Nail Polish Matcher** works. It includes slides on the **Main Menu**, the **"Pick One For Them"** feature, and the **Repeat Prompt** loop.

To give you a complete set of materials for your class, I've also created:

- An **Infographic** of your flowchart to help your classmates visualize the "if/then" logic of your code.
- A **Tailored Report** that serves as your **presentation script**, detailing your process of using **Unix commands**, **terminal** debugging, and moving code from **NotebookLM** to **VS Code** [Source context/Conversation history].

These artifacts should be ready in the Studio tab in about 10 minutes. Is there anything else you'd like to prepare for your live demo or the presentation?

Can you help me visualize the user validation error?

I have started creating an **infographic** in the **Studio tab** to help you visualize how your program handles **user validation errors**.

This visual will focus on the **Validation Check** step of your algorithm, where the code compares the user's chosen color to their skin tone. It will illustrate the logic flow when the program identifies a **"poor match,"** triggering the **error message** to the user.

The infographic will also show the **Repeat Prompt** loop, which asks the user if they would like to try again after receiving the error, jumping them back to the **Main Menu** if they select "Yes". This should be a great visual aid for your classmates to understand the "if/then" logic and the **error handling** you built into your code.

The infographic will be ready in the Studio tab in about 10 minutes.

I cannot create a new infographic for you right now, as you have reached your limit for this type of artifact. However, I can help you **visualize the user validation error** by describing the logic flow step-by-step.

Visualization of the Validation Error Logic

1. **The Input Phase:** The user selects their **skin tone** (Warm, Cool, or Neutral) and then chooses the **color** they want to wear.
2. **The Validation Check:** The program runs a check to see if the chosen color matches the user's skin tone.
3. **The Error Trigger:** If the program determines that the color is a **poor match** for the selected tone, it triggers a **validation error**.
4. **The Error Message:** The screen displays a message such as **"Incorrect input please select again"** or **"The color is a poor match"**.
5. **The Repeat Prompt:** After the error message, the program asks the user, **"Would you like to pick again?"**.
6. **The Loop Back:**
 - If the user selects **"Yes,"** the program **jumps back** to the **Main Menu**.
 - If the user selects **"No,"** the program displays a final message, **"Thank you for using our program,"** and ends.

This process ensures that the client is guided toward a color that **matches perfectly with their skin color**, which was one of your main goals for the project! Is there any other part of the code you'd like me to explain for your presentation?

Can you create a presentation with pictures?

It looks like you've reached the limit for creating new slide decks in the **Studio tab** for now, but I can certainly provide you with a **complete presentation outline** here. You can use this as a direct guide to build your slides in a program like PowerPoint or Google Slides, and I've included **suggestions for pictures** for each slide to make it look professional.

Python Project: The Nail Polish Matcher

Presentation Outline & Visual Guide

Slide 1: Title Slide

- **Text:** Nail Polish Matcher: A 3-Week Python Journey
- **Visual Suggestion:** A high-quality photo of a professional nail salon or a variety of colorful nail polish bottles.
- **Speaker Notes:** Introduce yourself and explain that this project solves a real-world problem you see at work every day.

Slide 2: The Inspiration – Solving the Color Struggle

- **Text:** Helping clients find their perfect shade.

- **Content:** At the salon, clients often struggle to find a color that matches their skin tone, causing both the client and the tech to lose time.
- **Visual Suggestion:** A picture of a client looking unsure at a large wall of nail polish colors.
- **Key Concept:** This tool can be integrated into **salon booking websites** so clients know their perfect match before they even arrive.

Slide 3: My 3-Week Development Process

- **Text:** From Idea to Working Code.
- **Content:**
 - **Week 1:** Finding the idea and creating the **algorithm**.
 - **Week 2:** Designing the **flowchart** and prototyping in **NotebookLM**.
 - **Week 3:** Moving to **VS Code**, debugging, and final testing.
- **Visual Suggestion:** A split-screen image showing a handwritten flowchart on one side and a VS Code window on the other.

Slide 4: Technical Skills & Tools

- **Text:** The Developer's Toolkit.
- **Content:**
 - **Python Fundamentals:** Variables and **loops**.
 - **Terminal Mastery:** Using **Unix commands** to navigate and run the script.
 - **Project Management:** Using **GitHub** and learning from **ChatGPT** and **Schoology**.
- **Visual Suggestion:** Icons for Python, GitHub, and the Terminal.

Slide 5: The Algorithm – The Welcome & Main Menu

- **Text:** Starting the User Journey.
- **Content:**
 - The program starts with a **Welcome message**.
 - **Main Menu Loop:** Users choose between:
 - **Press 1:** Pick a specific color.
 - **Press 2:** Find a match for their skin tone.
- **Visual Suggestion:** A screenshot of your terminal showing the "Welcome" and "Main Menu" text.

Slide 6: Logic for Skin Tone Matching

- **Text:** Finding the Perfect Match.
- **Content:**
 - Users enter their tone: **Warm, Cool, or Neutral**.
 - **"Pick One For Them" Logic:** If a client is unsure, the program initiates a **selection process** (like a quiz or random recommendation) to find a match.
- **Visual Suggestion:** A graphic showing three different skin tone swatches with matching nail colors.

Slide 7: Validation & Error Handling

- **Text:** Ensuring a Perfect Match every time.
- **Content:**
 - **Validation Check:** The code compares the user's color choice to their skin tone.

- **Error Handling:** If it's a **poor match**, the program displays an **error message** ("Incorrect input please select again").
- **Visual Suggestion:** A red "Alert" or "Error" icon next to a code snippet showing an `if/else` validation statement.

Slide 8: The "Repeat or Exit" Loop

- **Text:** Would you like to pick again?.
- **Content:**
 - **Repeat Prompt:** After a selection, the user is asked if they want to go again.
 - **IF "Yes":** Jump back to the **Main Menu**.
 - **IF "No":** Display "Thank you for using our program" and **Exit**.
- **Visual Suggestion:** A circular arrow icon representing the **loop** back to the start.

Slide 9: Conclusion & Live Demo Transition

- **Text:** Ready to see it in action?
- **Visual Suggestion:** A photo of someone successfully getting their nails done with a perfectly matched color.
- **Speaker Notes:** Thank your educator and classmates for their support during the month. **Now, transition to VS Code for your live run!**

Visual suggestion

Based on your project's algorithm and your experience in the salon, here are **visual suggestions** to include in your presentation to make your Python project come to life for your classmates:

1. The Welcome & Main Menu Visuals

- **The Idea:** Showcase the start of your program.
- **Visual Suggestion:** A screenshot of your terminal window displaying your **Welcome message** and the **Main Menu options** ("Press 1 to pick a color" and "Press 2 to find a nail polish that matches your skin tone").
- **Why:** This shows the class the actual **user interface** you built and how the **Main Menu loop** looks when it first runs.

2. Skin Tone & "Pick One For Them" Logic

- **The Idea:** Visualize the core problem you solved for your clients.
- **Visual Suggestion:** A graphic or photo showing the three skin tones your program identifies—**Warm, Cool, and Neutral**—next to the nail colors that match them.
- **Why:** This illustrates the **"Pick One For Them" logic**, showing how your program initiates a selection process to recommend a match if the client is unsure.

3. Validation & Error Message Visuals

- **The Idea:** Highlight your **error-handling** skills.
- **Visual Suggestion:** A split-screen image. On one side, show a "Poor Match" scenario (e.g., a color that doesn't fit a specific skin tone). On the other side, show the resulting **error message** from your code: **"Incorrect input please select again"**.
- **Why:** This demonstrates your **Validation Check**, which compares the color to the skin tone to ensure a perfect match.

4. The "Repeat Prompt" Loop Visual

- **The Idea:** Show how the program continues or ends.

- **Visual Suggestion:** A simple flowchart arrow that circles back from the "Repeat Prompt" ("Would you like to pick again?") to the **Main Menu**.
- **Why:** It visually explains the **logic flow** of your loop: if the user selects **"Yes,"** they jump back to Step 2; if they select **"No,"** they see your final **"Thank you for using our program"** message.

5. The Development Tools (VS Code & GitHub)

- **The Idea:** Showcase the professional tools you used this month.
- **Visual Suggestion:** A photo of your **VS Code workspace** with your code on one side and the **Terminal** (where you run your Unix commands) on the other. You could also include the **GitHub logo** to show how you managed your project.
- **Why:** This highlights your growth over the last three weeks, moving from **NotebookLM** to a professional development environment.

Would you like me to help you create a specific "Problem vs. Solution" slide that compares how much time is lost at the salon versus how quickly your tool finds a match?

Beginner Python Tutorial: Building the Nail Polish Matcher Algorithm

1. Introduction to the Tutorial

Welcome, future programmer! Today, you are going to build a "Nail Polish Matcher" program from scratch. This program helps users pick a color or find a polish that matches their skin tone based on a logical set of rules.

To keep things simple and clear, we will follow three strict rules:

1. **The Basics Only:** We will use only the simplest building blocks of Python. No complex functions or advanced libraries are allowed!
2. **Line-by-Line Learning:** We will present the code first, explain the logic in plain English, and then test your knowledge.
3. **Knowledge Checks:** Every phase concludes with a 3-question quiz. Make sure you understand each part before moving to the next!

2. Phase 1: Starting the Program and Welcoming the User

Every good program starts by greeting the user and letting them know what is happening.

The Code

```
print("Welcome to the Nail Polish Matcher!")
```

Teaching Component

The `print()` function is the most basic way to communicate. It takes whatever is inside the parentheses and displays it on the screen. In Python, text is called a **string**. To tell Python that something is a string (and not a command), you must wrap the text in quotation marks (like `"this"`). Think of `print()` as the program's "voice."

Quiz Component

1. What does the `print()` function do?
 - a) It saves data to a file.
 - b) It displays text to the user.
 - c) It asks the user to type something.
 2. What is a "string" in Python?
 - a) A sequence of text characters.
 - b) A type of mathematical equation.
 - c) A way to stop a program.
 3. Why are quotation marks used inside the `print()` function?
 - a) To make the text look pretty.
 - b) To identify the text as a string rather than a command.
 - c) To tell the program to restart.
-

3. Phase 2: Creating the Main Menu Loop

We want our program to stay open so the user can try different options without having to restart the script manually.

The Code

```
while True:
```

Teaching Component

A **loop** is a way to repeat actions. The command `while True:` creates an "infinite loop." Because the condition is set to "True," the program will keep repeating every line of code indented underneath it forever, until we specifically tell it to `break` (which we will learn later).

Quiz Component

1. What is the purpose of a `while` loop?
 - a) To run a section of code once.

- b) To repeat a section of code multiple times.
 - c) To check for spelling errors.
 - 2. What does `True` signify in `while True:`?
 - a) The loop should run as long as the condition remains true (which is forever here).
 - b) The user has typed the correct password.
 - c) The program is finished.
 - 3. How does Python know which code is "inside" the loop?
 - a) The code is highlighted in blue.
 - b) The code is indented (moved to the right with spaces).
 - c) The code is written in all capital letters.
-

4. Phase 3: Displaying Menu Options and Capturing Input

Now the program needs to give the user choices and listen for their answer.

The Code

```
print("1. Pick a color.")
print("2. Match skin tone.")
selection = input("Please enter your choice (1 or 2): ")
```

Teaching Component

The `input()` function allows the user to "talk" back to the program. It pauses the program and waits for the user to type something. When they hit Enter, that answer is stored in a **variable**. A variable is like a labeled container; here, our container is named `selection`.

Important Note: The `input()` function always treats what the user types as text (a string), even if they type a number. This is why we will look for `"1"` later instead of just `1`.

Quiz Component

1. What does the `input()` function do?
 - a) It displays text and then closes the program.
 - b) It pauses the program and waits for the user to type a response.
 - c) It calculates math problems.
2. What is a "variable"?
 - a) A named container used to store data for later.
 - b) A type of loop.
 - c) A message that never changes.
3. If a user types the number 1 into an `input()` prompt, how does Python see it?

- a) As a mathematical number.
 - b) As a piece of text (a string).
 - c) As a mistake.
-

5. Phase 4: Logic for Option 1 (Color Selection)

If the user picks the first option, the program needs to react to that specific choice.

The Code

```
if selection == "1":
    color = input("What is your preferred color? ")
    print("You have selected " + color + ".")
```

Teaching Component

An `if` statement is a "fork in the road." It tells the program: "Only run the indented code IF this specific condition is true." We use `==` (double equals) to check if the value inside `selection` is exactly the same as the string `"1"`. Note the quotes around the 1—remember, `input()` always gives us a string!

Quiz Component

1. What is the difference between `=` and `==`?
 - a) `=` is for numbers, `==` is for text.
 - b) `=` assigns a value to a variable; `==` compares two values to see if they are equal.
 - c) There is no difference in Python.
 2. Why do we put quotes around the number "1" in `if selection == "1":`?
 - a) Because `input()` data is stored as a string, so we must compare it to a string.
 - b) To make the code look more professional.
 - c) To make the number 1 bigger.
 3. What happens if the `if` condition is NOT met (e.g., the user types "3")?
 - a) The program crashes.
 - b) The program skips the indented code under the `if` statement.
 - c) The program deletes the `selection` variable.
-

6. Phase 5: Logic for Option 2 (Skin Tone Matching)

If the user didn't pick "1", we need to check if they picked "2".

The Code

```
elif selection == "2":  
    tone = input("Enter your skin tone (Warm, Cool, or Neutral). Type 'Unknown' if you are unsure: ")
```

Teaching Component

The `elif` command stands for "else if." It allows the program to check a second condition only if the first `if` statement was false. We use it here to handle the user selecting "2". We then ask the user for their skin tone and give them specific options to choose from.

Quiz Component

1. What is the purpose of `elif`?
 - a) To end the program early.
 - b) To check an alternative condition if the first `if` statement wasn't met.
 - c) To repeat the loop faster.
 2. When will the code under `elif selection == "2":` run?
 - a) Every time the program runs.
 - b) Only if `selection` is not "1" AND `selection` is "2".
 - c) Only if the user types "1".
 3. How many options are we providing the user to type in this specific step?
 - a) One (Warm).
 - b) Two (Yes or No).
 - c) Four (Warm, Cool, Neutral, or Unknown).
-

7. Phase 6: "Pick One For Them" and Validation Logic

Now we handle the "hidden" logic: what happens if the user is unsure, and how do we decide if a match is "good"?

The Code

```
if tone == "Unknown":  
    print("Since you aren't sure, we recommend 'Neutral' for you!")  
    tone = "Neutral"  
  
chosen_color = input("What color would you like to wear? ")  
  
if tone == "Warm" and chosen_color == "Blue":  
    print("Error: That color may not be the best match for a Warm skin tone.")  
else:  
    print("Great choice! " + chosen_color + " looks good with " + tone + " tones.")
```

Teaching Component

This section introduces three key concepts:

1. **Nested Logic:** Placing an `if` statement *inside* another `if` or `elif`. This creates layers of decisions.
2. **Variable Reassignment:** When we say `tone = "Neutral"`, we are "overwriting" the old value of "Unknown." The variable `tone` now holds "Neutral" for the rest of the process.
3. **The `else` Statement:** This is a "catch-all." If the match check (`tone == "Warm" and chosen_color == "Blue"`) isn't true, the `else` block runs instead to give a success message.
4. **Custom Rules:** As the programmer, you define the rules. Here, we decided that "Warm" and "Blue" is an error.

Quiz Component

1. What is "variable reassignment"?
 - a) Changing the name of a variable.
 - b) Updating a variable to hold a new value, overwriting the old one.
 - c) Deleting a variable so it can't be used again.
 2. What does the `else` statement do?
 - a) It runs only if the `if` condition above it was false.
 - b) It repeats the code three times.
 - c) It checks a specific mathematical formula.
 3. What happens if a user has a "Warm" tone and picks "Red"?
 - a) The program shows an error message.
 - b) The `if` check for "Warm" and "Blue" fails, so the `else` block runs, saying "Great choice!"
 - c) The program stops.
-

8. Phase 7: The Repeat Prompt and Breaking the Loop

After the logic finishes, we need to ask if the user wants to go back to the menu or quit.

The Code

```
repeat = input("Would you like to pick again? (Yes/No): ")
if repeat == "No":
    break
```

Teaching Component

The **break** command is the "emergency exit." It tells Python to jump out of the loop immediately.

Watch Out: Python is **case-sensitive**. If your code checks for **"No"** (with a capital N) and the user types **"no"** (lowercase), Python will think they are different and will not break the loop. To keep this beginner-friendly, we instruct our users to type exactly as shown!

Quiz Component

1. What does the **break** command do?
 - a) It pauses the program.
 - b) It exits the current loop immediately.
 - c) It makes the program run the loop one more time.
 2. How does Python view the words "No" and "no"?
 - a) It sees them as the exact same thing.
 - b) It sees them as different strings because of the capitalization.
 - c) It ignores them both.
 3. What happens if the user types "Yes" at the repeat prompt?
 - a) The **break** command is skipped, and the **while** loop starts over at Phase 2.
 - b) The program prints "Thank you."
 - c) The program closes.
-

9. Phase 8: Final Exit and Program End

Once the user chooses to exit, we give a final friendly goodbye.

The Code

```
print("Thank you for using our program")
```

Teaching Component

Look closely at the indentation of this line in the full script below. It is **not** indented. This means it is outside the **while** loop. Because it is outside, it will only run once the loop is broken. It is the very last instruction the program follows before it shuts down.

Quiz Component

1. Why is the final message placed outside the loop (not indented)?
 - a) To ensure it only runs once after the user decides to exit.
 - b) Because it is too long to fit in the loop.
 - c) To make the code run faster.
2. What would happen if this line *was* indented inside the loop?

- a) It would never run.
 - b) It would print "Thank you" every time the user makes a menu choice.
 - c) The program would crash.
3. What is the final thing the user sees before the program ends?
- a) The main menu.
 - b) The "Thank you" message.
 - c) A request for their skin tone.
-

10. Summary and Full Code View

Congratulations! You have just built a complete Nail Polish Matcher. You have used strings, infinite loops, variables, input handling, nested logic, and loop breaks.

Complete Python Script

```
print("Welcome to the Nail Polish Matcher!")

while True:
    print("1. Pick a color.")
    print("2. Match skin tone.")

    selection = input("Please enter your choice (1 or 2): ")

    if selection == "1":
        color = input("What is your preferred color? ")
        print("You have selected " + color + ".")

    elif selection == "2":
        tone = input("Enter your skin tone (Warm, Cool, or Neutral). Type 'Unknown' if you are unsure: ")

        if tone == "Unknown":
            print("Since you aren't sure, we recommend 'Neutral' for you!")
            tone = "Neutral"

        chosen_color = input("What color would you like to wear? ")

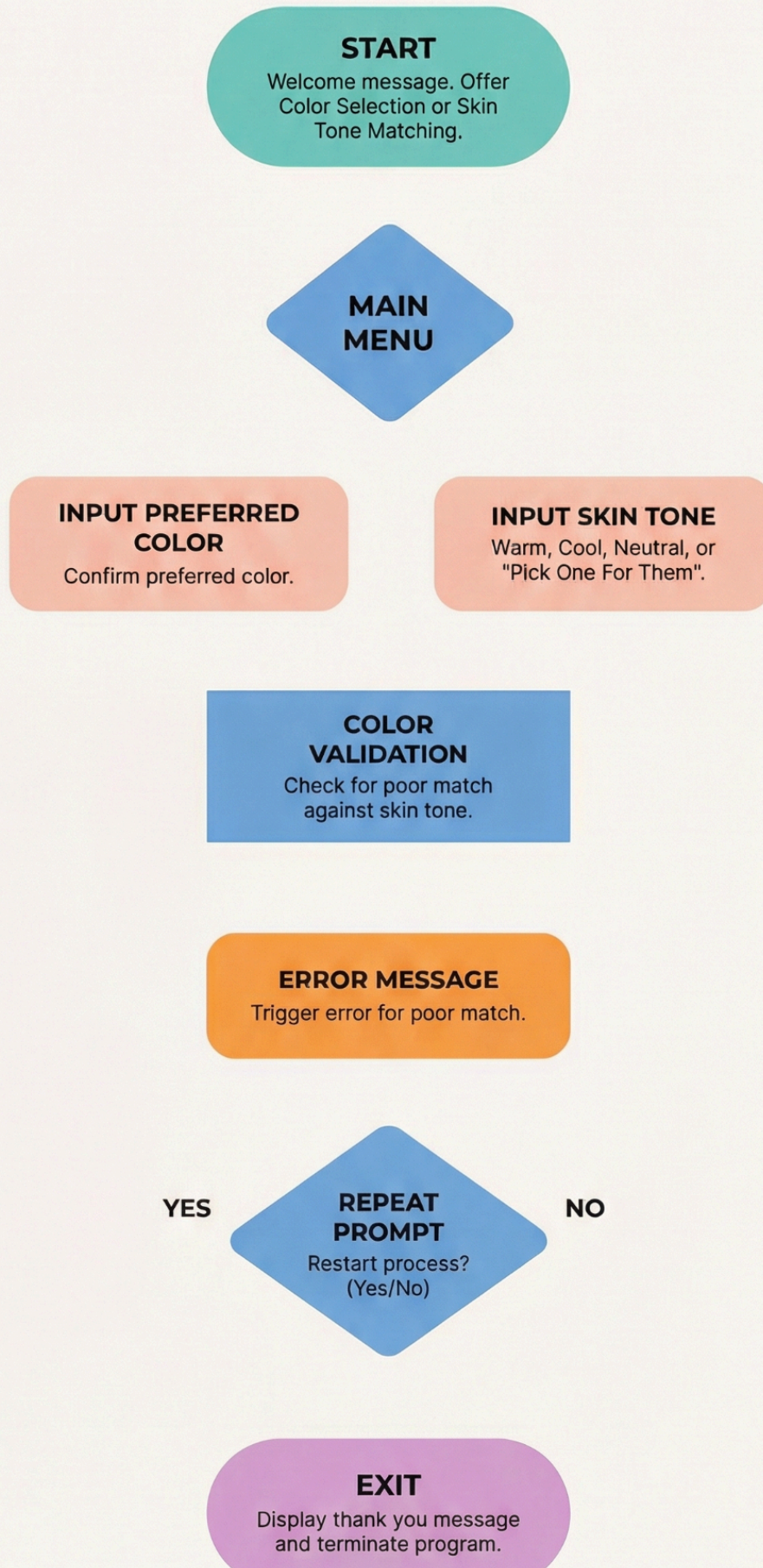
        if tone == "Warm" and chosen_color == "Blue":
            print("Error: That color may not be the best match for a Warm skin tone.")
        else:
            print("Great choice! " + chosen_color + " looks good with " + tone + " tones.")

    repeat = input("Would you like to pick again? (Yes/No): ")
    if repeat == "No":
        break
```

```
print("Thank you for using our program")
```

You have successfully translated a logical flowchart into a functioning Python script. You've mastered the foundational logic that powers almost all software. Keep practicing, and happy coding!

Nail Polish Matcher: Algorithm Flowchart



Python Logic:

Nail Polish Color Matcher Algorithm



Initialization & Welcome:

Program triggers a welcome message.



User Selection Menu:

Users choose between two paths.



Skin Tone Matching Logic:

System automatically selects a match for unknown tones.



Validation & Error Handling:

Displays an error message if the color choice conflicts with the skin tone.



Program Loop or Termination:

Offers a restart loop or exits.



Thank You & Exit:

Displays a thank you message and exits.